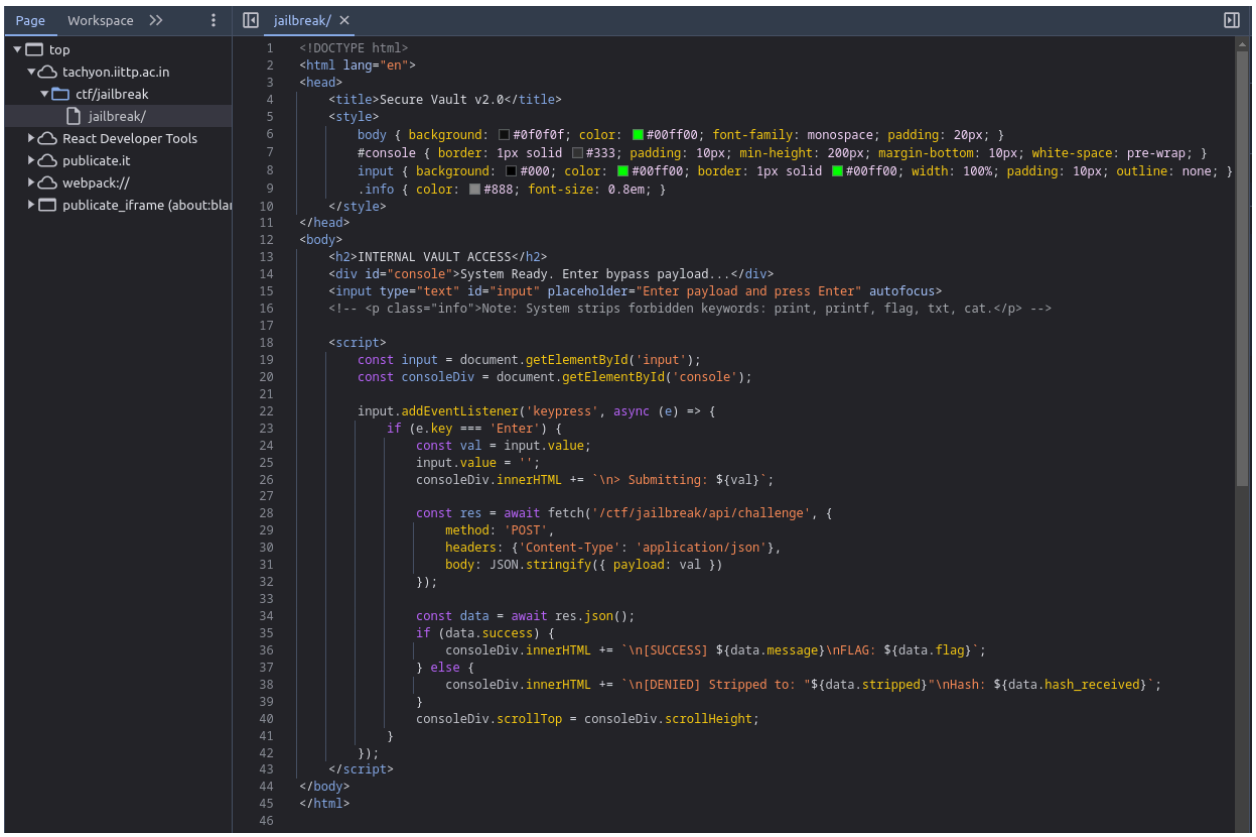
GLITCHED

Challenge Description:

Can you break into the vault?
https://tachyon.iittp.ac.in/ctf/jailbreak/

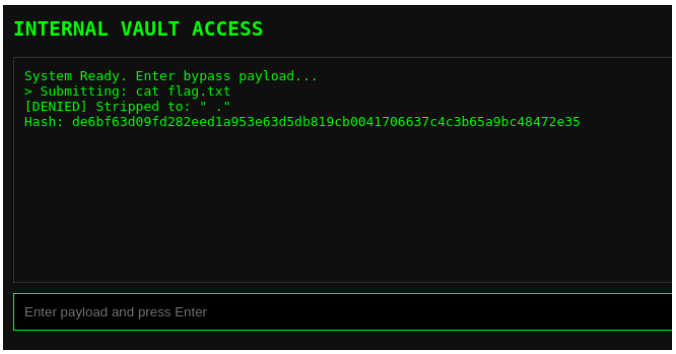
Solution:

This is a challenge where you have to bypass a common input replacement/stripping algorithm that is in place in the backend so that users don't inject scripts as inputs….
Here, if we try to inspect the console, we see that:



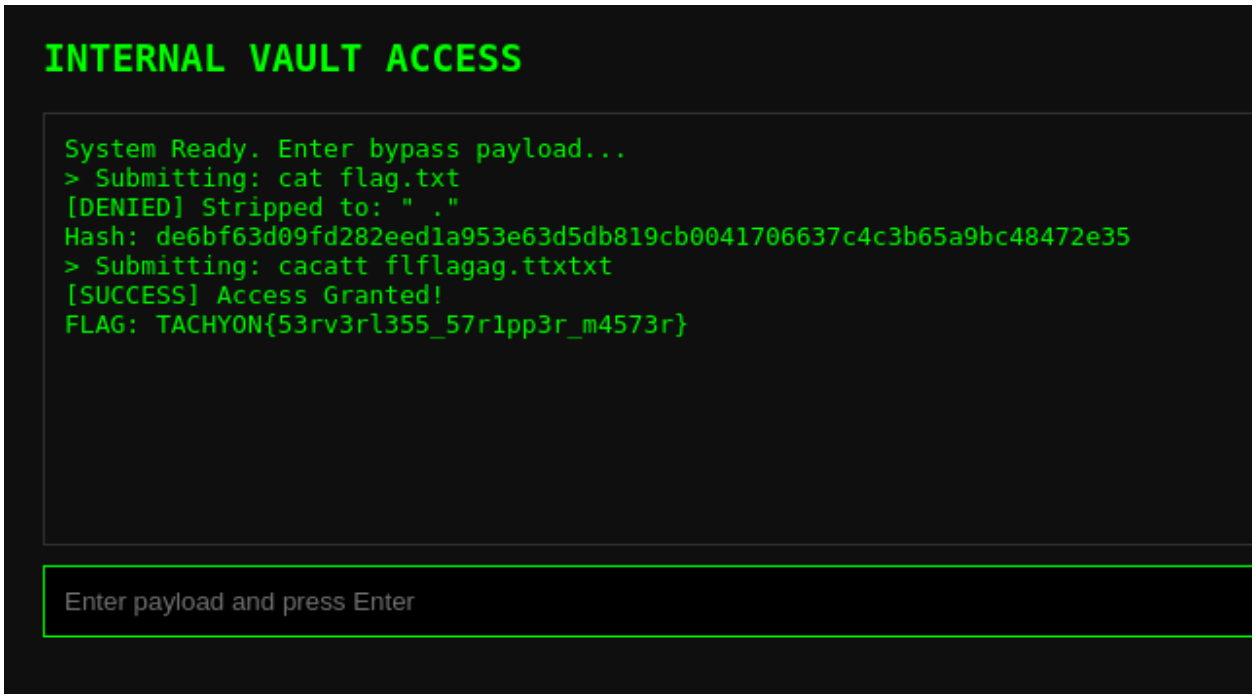

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Secure Vault v2.0</title>
    <style>
        body { background: #0f0f0f; color: #00ff00; font-family: monospace; padding: 20px; }
        #console { border: 1px solid #333; padding: 10px; min-height: 200px; margin-bottom: 10px; white-space: pre-wrap; }
        input { background: #000; color: #00ff00; border: 1px solid #00ff00; width: 100%; padding: 10px; outline: none; }
        .info { color: #888; font-size: 0.8em; }
    </style>
</head>
<body>
    <h2>INTERNAL VAULT ACCESS</h2>
    <div id="console">System Ready. Enter bypass payload...</div>
    <input type="text" id="input" placeholder="Enter payload and press Enter" autofocus>
    <!-- <p class="info">Note: System strips forbidden keywords: print, printf, flag, txt, cat.</p> -->

    <script>
        const input = document.getElementById('input');
        const consoleDiv = document.getElementById('console');

        input.addEventListener('keypress', async (e) => {
            if (e.key === 'Enter') {
                const val = input.value;
                input.value = '';
                consoleDiv.innerHTML += `\n> Submitting: ${val}`;

                const res = await fetch('/ctf/jailbreak/api/challenge', {
                    method: 'POST',
                    headers: {'Content-Type': 'application/json'},
                    body: JSON.stringify({ payload: val })
                });

                const data = await res.json();
                if (data.success) {
                    consoleDiv.innerHTML += `\n[SUCCESS] ${data.message}\nFLAG: ${data.flag}`;
                } else {
                    consoleDiv.innerHTML += `\n[DENIED] Stripped to: "${data.stripped}"\nHash: ${data.hash_received}`;
                }
                consoleDiv.scrollTop = consoleDiv.scrollHeight;
            }
        });
    </script>
</body>
</html>
```

In the comment, we see that print, printf, flag etc are going to be stripped… this gives us the idea that we can try and cat the flag through the input box:



We see that cat flag.txt has been completely stripped all three strings - cat, flag and txt, now we can give padded inputs like fl**flag**ag so that the bolded **flag** gets stripped but the fl and ag then remain to give back flag, this way we craft our payload -

cacatt flflagag.ttxtxt and voila!



Flag:
TACHYON{53rv3rl355_57r1pp3r_m4573r}